

Funcionamiento de la aplicación

Proyecto educativo de Física desarrollado con Vue.js y Vite

1. Descripción general

La aplicación es una plataforma web educativa para consultar conceptos fundamentales de física. Presenta temas teóricos, imágenes, fórmulas matemáticas, ejemplos y ejercicios resueltos mediante una interfaz interactiva.

La información se almacena localmente en archivos JSON y módulos JavaScript. Estos archivos funcionan como una base de datos estática: contienen los registros del contenido y los componentes de Vue los consultan, procesan y muestran en pantalla.

Idea central: los archivos de datos separan el contenido educativo de la interfaz. De esta manera, es posible agregar o modificar temas y ejercicios sin cambiar la estructura principal de los componentes.

2. Fuentes de información

`src/assets/fisica.json` : índice general de temas

Es el archivo principal del catálogo. Contiene una lista de objetos con los campos `title`, `image` y `description`. Cada objeto representa un tema disponible en la aplicación.

`src/assets/topicDetails.js` : contenido teórico ampliado

Contiene un objeto cuyos identificadores son los títulos de los temas. Cada entrada incluye el título, una imagen y una explicación extensa en HTML, con fórmulas, tablas, ejemplos y recursos visuales.

`src/assets/exercisesData.js` : banco de ejercicios

Organiza los ejercicios por tema. Cada tema contiene una lista de objetos con un `enunciado` y una `solucion`. El contenido puede incluir HTML, imágenes y expresiones matemáticas.

3. Flujo de datos y renderizado

```
fisica.json
```

```
↓
```

```
ModalIndex.vue → muestra el índice de temas
```

```
↓  
CardComponent.vue → busca y muestra título, imagen y descripción  
↓  
TopicDetail.vue → carga la explicación completa desde  
topicDetails.js  
↓  
SolvedExercisesView.vue → carga los ejercicios desde  
exercisesData.js
```

Inicio y selección de temas

`HomeView.vue` incorpora el componente `ModalIndex.vue`. Este importa `fisica.json` y recorre sus registros con `v-for` para crear una lista de botones. Al seleccionar un tema, el título se envía como parámetro a Vue Router.

Consulta de la tarjeta del tema

`CardComponent.vue` recibe el título desde la ruta, busca el registro correspondiente en `fisica.json` y guarda el resultado en `topic`. Después renderiza la imagen, el título y la descripción. También ofrece navegación hacia la explicación detallada.

Visualización de la teoría

`TopicDetail.vue` utiliza `getTopicDetails()` para localizar el contenido ampliado en `topicDetails.js`. Ese contenido se inserta mediante `v-html`. Posteriormente, las expresiones delimitadas por `$...$` o `$$...$$` se convierten en fórmulas visuales usando KaTeX.

Visualización de ejercicios

`SolvedExercisesView.vue` llama a `getExercises()` para obtener la colección asociada al tema. Para cada ejercicio muestra el enunciado y la solución. También procesa las fórmulas con KaTeX y permite ampliar las imágenes mediante `ImageLightbox.vue`.

4. Relación entre archivos

Archivo	Responsabilidad	Resultado en la aplicación
<code>fisica.json</code>	Almacenar el catálogo básico de temas.	Índice, títulos, imágenes y descripciones breves.
<code>ModalIndex.vue</code>	Recorrer el catálogo y gestionar la selección.	Lista de temas en la pantalla inicial.
<code>CardComponent.vue</code>	Buscar y presentar un registro del JSON.	Tarjeta resumen del tema.
<code>topicDetails.js</code>	Almacenar las lecciones completas.	Contenido teórico detallado.
<code>TopicDetail.vue</code>	Consultar y renderizar la teoría.	Página de explicación del tema.
<code>exercisesData.js</code>	Almacenar problemas y soluciones.	Banco de ejercicios resueltos.
<code>SolvedExercisesView.vue</code>	Consultar y presentar los ejercicios.	Enunciados, soluciones, fórmulas e imágenes.
<code>KaTeX</code>	Convertir sintaxis matemática en HTML visual.	Fórmulas correctamente formateadas.

5. Conclusión

La aplicación sigue una arquitectura sencilla basada en datos locales. `fisica.json` actúa como catálogo principal; `topicDetails.js` amplía la información teórica; y `exercisesData.js` aporta la práctica. Los componentes Vue funcionan como la capa que consulta esos datos y los transforma en vistas interactivas.

Este diseño facilita el mantenimiento del proyecto, permite reutilizar los componentes y hace posible ampliar el contenido educativo agregando nuevos registros con la misma estructura.